Chapter-5

Pattern Recognition in Remote Sensing

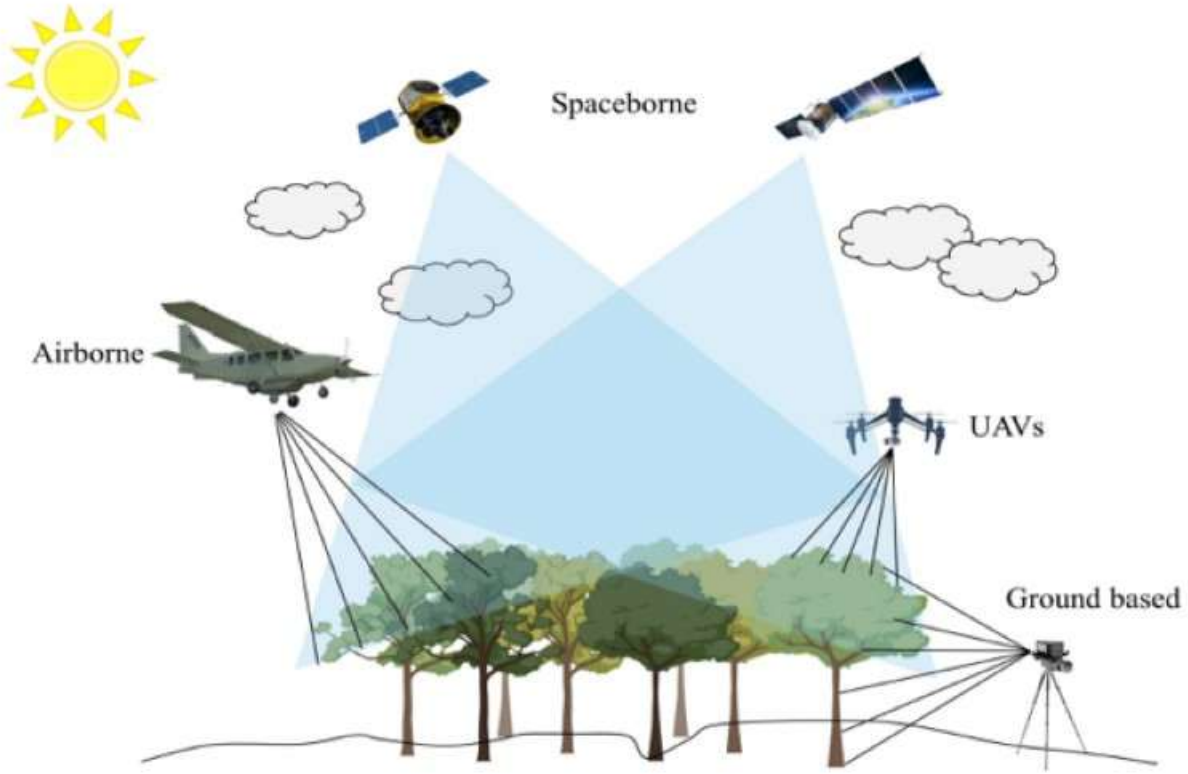
2025-12-22

Yagya Raj Pandeya, Ph.D.

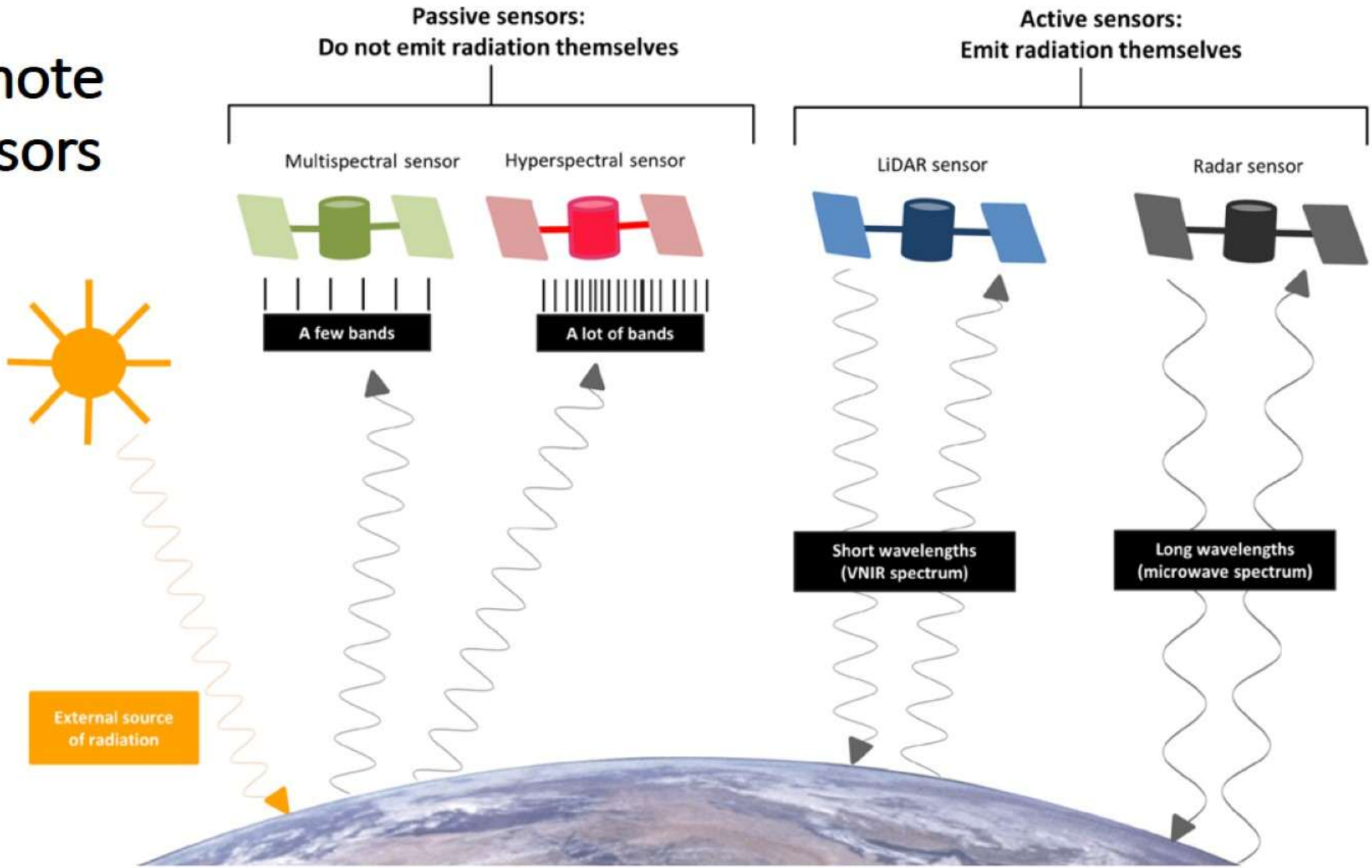
Assistant Prof., Dept. of AI, KU
Member, National AI Policy, MoCIT
Team Lead, National AI Strategy, EGB
CEO and Founder, Guru Technology
Editor in Chief, IJICTDC Int'l Journal
Lead Researcher, AISSR Lab, KU, G-SARP Lab, Pulchowk
AI Consultant and Educator

Remote Sensing Platforms

- Remote sensing is the science of acquiring information about Earth (and other planets) ***without making physical contact*** with the object of interest.
- We use specialized instruments like cameras, lasers, and radar to capture data.



Remote Sensors



Types of Sensors

The type of sensor determines the kind of data collected:

- **Optical Sensors:** Capture visible, near-infrared, and shortwave infrared radiation to create images resembling what we see.
- **Multispectral Sensors:** Capture data in multiple discrete bands of the electromagnetic spectrum, providing information beyond the visible range.
- **Hyperspectral Sensors:** Capture data in ***hundreds of contiguous bands***, allowing for detailed analysis of material properties.
- **Radar Sensors:** Emit radio waves and measure the reflected energy to create images independent of sunlight, useful for studying through clouds or for **vegetation analysis.**
- **LiDAR (Light Detection and Ranging):** Uses lasers to measure distances, creating highly accurate 3D models of the terrain.

Data acquisition and representation

- Remote sensing relies on acquiring data about the Earth's surface, atmosphere, and oceans from a distance.
- Sensors are mounted on various platforms to collect remote sensing data:
  - **Satellites:** Orbiting the Earth, satellites offer global coverage and frequent revisits for monitoring changes. (e.g., Landsat, Sentinel)
  - **Airplanes:** Provide high-resolution data collection for specific areas of interest.
  - **Unmanned Aerial Vehicles (UAVs/Drones):** Offer flexibility and high-resolution data capture for smaller areas.
  - **Ships and Ground-based Sensors:** Used for specific applications like oceanographic studies or collecting high-resolution ground data.

Applications

- **Environmental monitoring:** Studying deforestation, tracking climate change, monitoring pollution levels.
- **Land management:** Mapping land cover, assessing soil moisture, monitoring agricultural crops.
- **Disaster management:** Mapping flood inundation, assessing earthquake damage, tracking wildfires.
- **Resource exploration:** Identifying potential mineral deposits, locating groundwater resources.
- **Urban planning:** Monitoring urban growth, analyzing land use patterns.

Image de-noising and super-resolution

- **Image de-noising:** process of removing noise from an image to improve its visual quality and enhance the accuracy of subsequent image analysis
- Common types of noise include Gaussian noise, salt-and-pepper noise, speckle noise, and Poisson noise.

Original Image

Denoised Image

De-noising Techniques

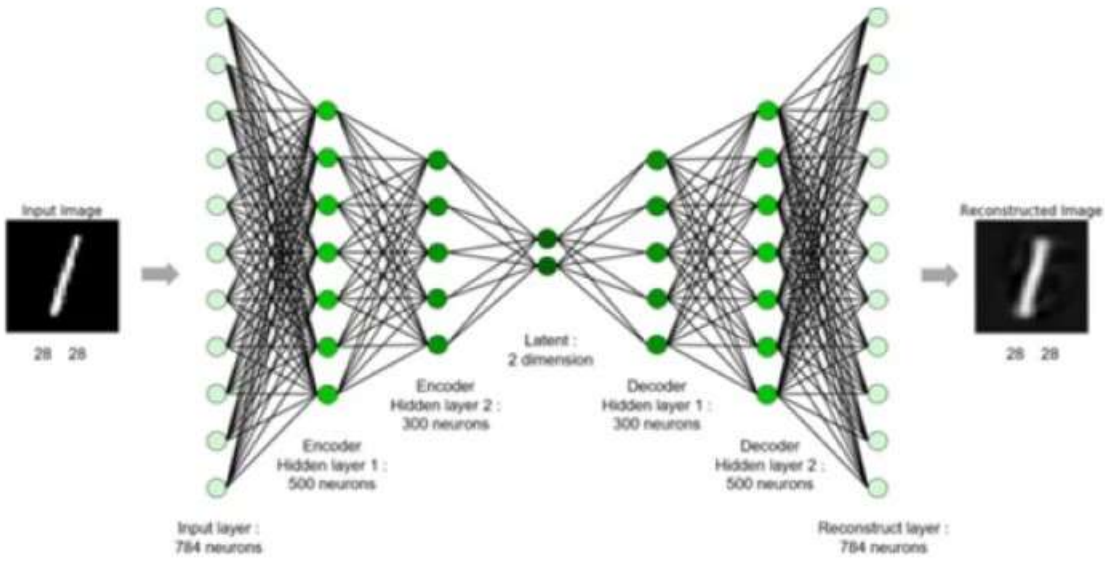
Some popular denoising techniques are:

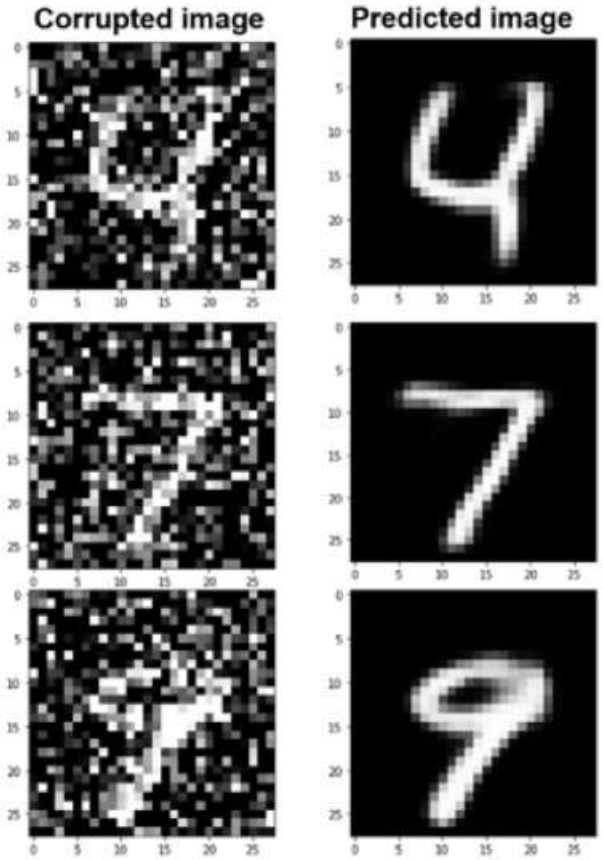
- **Spatial Filtering:** Techniques such as Gaussian filtering, median filtering, and bilateral filtering apply convolution masks to smooth or blur the image while preserving edges.
- **Wavelet Transform:** Wavelet-based denoising decomposes the image into different frequency bands, allowing for noise reduction in specific frequency ranges.
- **Non-local Means (NLM):** NLM denoising exploits the redundancy of image patches to estimate noise-free pixel values by averaging similar patches.
- **Deep Learning:** Convolutional neural networks (CNNs) trained specifically for image denoising can learn complex noise patterns and perform effective denoising.

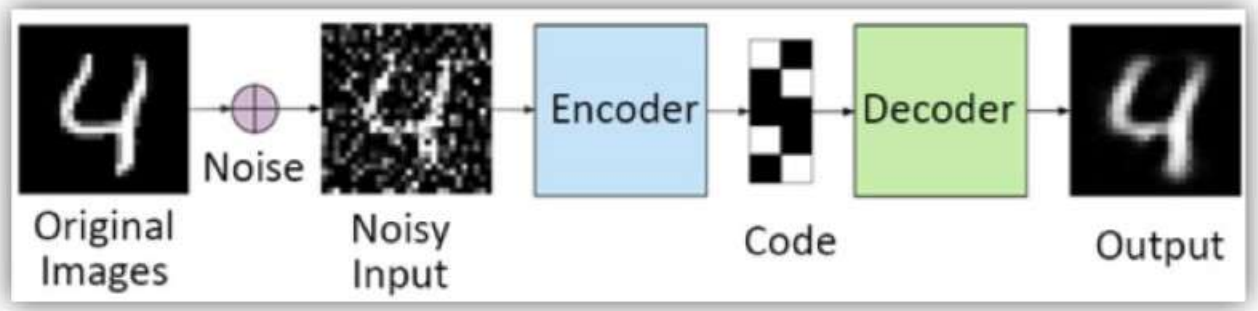
Evaluating Denoising Performance

- peak signal-to-noise ratio (PSNR),
- structural similarity index (SSIM),
- mean squared error (MSE),
- visual inspection by human observers.

De-noising auto-encoders







Denoising Autoencoder using Tensorflow

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt

# Load and preprocess MNIST dataset
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))

# Add random noise to the images
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)

# Clip pixel values to [0, 1]
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# Define the convolutional denoising autoencoder model
input_img = layers.Input(shape=(28, 28, 1))
```

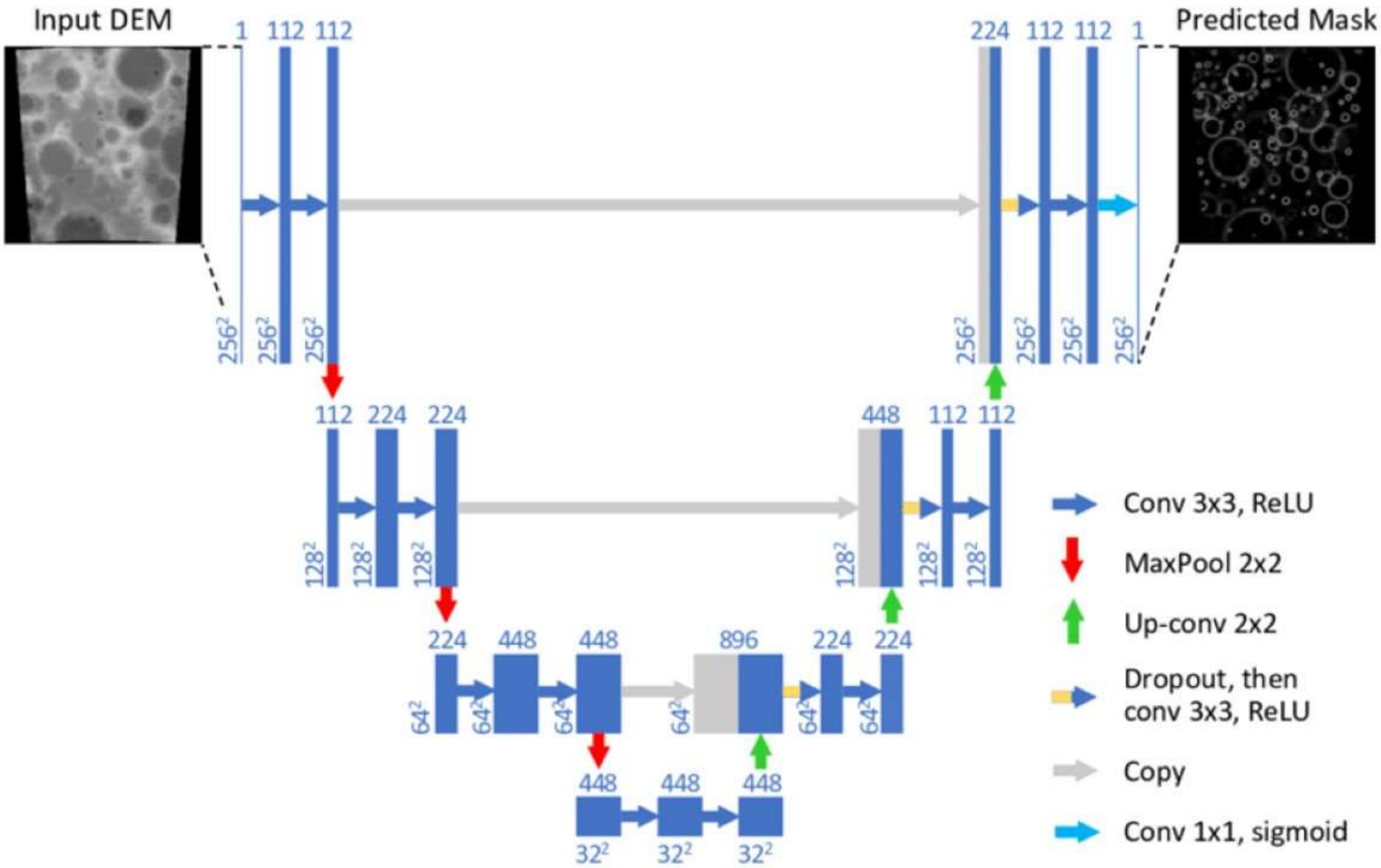
Denoising Autoencoder using Tensorflow

```
# Encoder
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = layers.MaxPooling2D((2, 2), padding='same')(x)

# Decoder
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = layers.UpSampling2D((2, 2))(x)
decoded = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

# Model
autoencoder = models.Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# Train the model
autoencoder.fit(x_train_noisy, x_train, epochs=10, batch_size=128,
                shuffle=True,validation_data=(x_test_noisy, x_test))
# Predict on noisy test images
decoded_imgs = autoencoder.predict(x_test_noisy[:10])
```

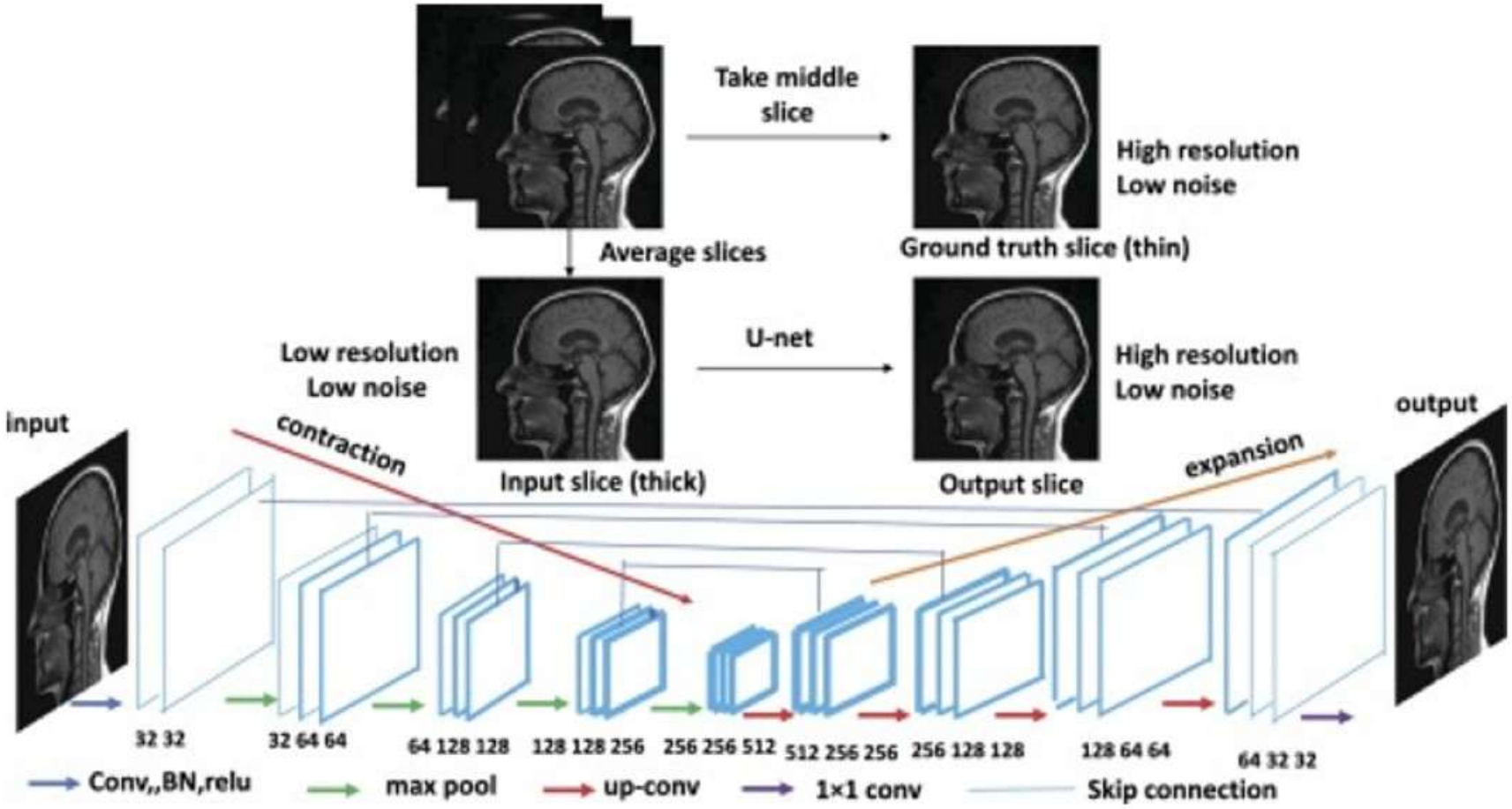
Denoising Autoencoder using Tensorflow

```
# Display original, noisy, and denoised images
n = 10
plt.figure(figsize=(20, 6))
for i in range(n):
    # Noisy
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
    plt.title("Noisy")
    plt.axis('off')
    # Denoised
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Denoised")
    plt.axis('off')
    # Original
    ax = plt.subplot(3, n, i + 1 + 2 * n)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

U-net


Input DEM
Predicted Mask
1 112 112
224 112 112 1
256²
112 224 224
448 112 112
128²
224 448 448 896 224 224
64²
448 448 448
32²
Conv 3x3, ReLU
MaxPool 2x2
Up-conv 2x2
Dropout, then conv 3x3, ReLU
Copy
Conv 1x1, sigmoid

Medical Images Super Resolution

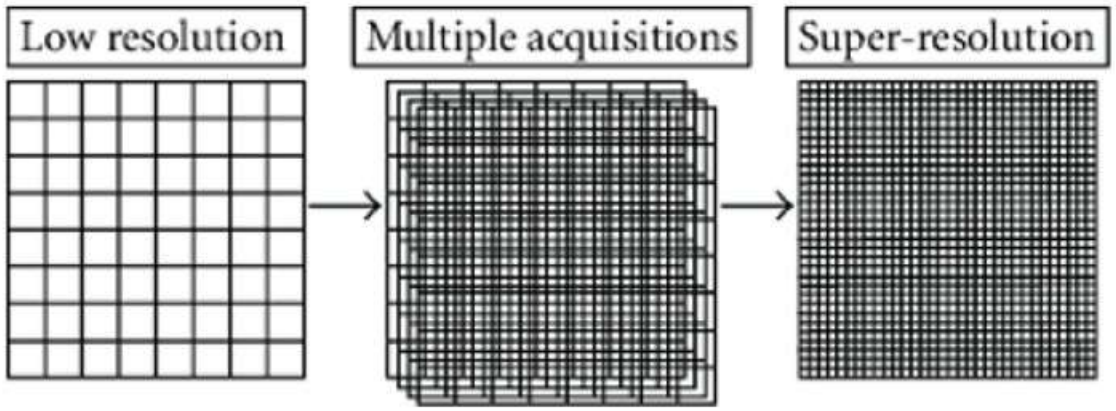


A Review of the Deep Learning Methods for Medical Images Super Resolution Problems

https://doi.org/10.1016/j.irbm.2020.08.004

Super-resolution

- Process of enhancing the resolution of an image to produce a higher-resolution version with improved visual quality and finer details.
- useful in scenarios where the original image has low resolution or limited spatial information
- **Applications:** medical imaging, satellite imaging, surveillance, digital photography, and video processing, where high-resolution images are crucial for accurate analysis and interpretation.



Ground Truth	Bicubic	Ours (ℓ_{pixel})	SRCNN	Ours (ℓ_{feat})
This Image	21.69 / 0.5840	21.66 / 0.5881	22.53 / 0.6524	21.04 / 0.6116

Super-resolution techniques

A. Single-Image Super-Resolution (SISR):

- **Upsampling Methods:** Traditional upsampling techniques, such as bicubic interpolation or nearest-neighbor interpolation, are used to increase the size of the image. However, these methods often result in blurry images and do not add new details.
- **Learning-Based Approaches:** Deep learning-based methods, particularly convolutional neural networks (CNNs), have shown significant advancements in SISR. These models learn to map low-resolution input images to high-resolution output images by exploiting inherent patterns and structures in the data.

B. Multi-Image Super-Resolution (MISR):

- **Multiple Image Fusion:** MISR techniques use information from multiple low-resolution images of the same scene to reconstruct a higher-resolution image. This can be achieved by aligning and combining multiple images to increase the spatial information and reduce noise.
- **Patch-Based Methods:** These methods divide the low-resolution images into overlapping patches, estimate the corresponding high-resolution patches, and then fuse them to generate the final high-resolution image.

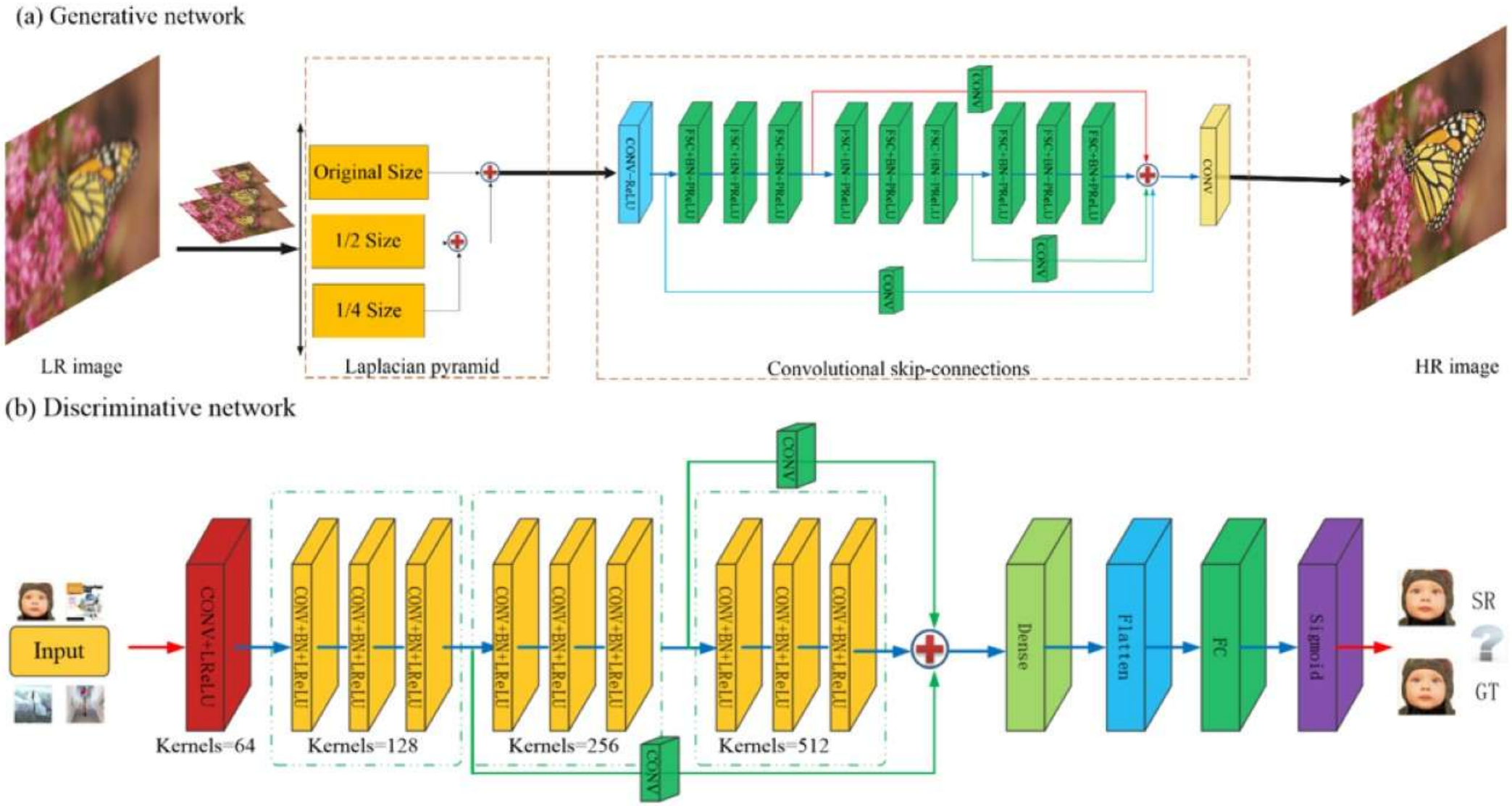
Super-resolution techniques

C. Deep Learning-Based Super-Resolution:

- **Single-Image Super-Resolution Networks (SISR-Nets):** SRCNN (Super-Resolution Convolutional Neural Network), VDSR (Very Deep Super-Resolution), and EDSR (Enhanced Deep Super-Resolution), are trained on pairs of low-resolution and high-resolution images to learn the mapping between them.
- **Generative Adversarial Networks (GANs):** GANs, such as SRGAN (Super-Resolution Generative Adversarial Network) and ESRGAN (Enhanced Super-Resolution Generative Adversarial Network)
- **Attention Mechanisms:** Some super-resolution networks incorporate attention mechanisms to focus on relevant image regions and allocate resources more efficiently during the super-resolution process.

Evaluation and Metrics: peak signal-to-noise ratio (PSNR), structural similarity index (SSIM), mean squared error (MSE), and perceptual quality metrics.

SR-GAN



https://www.mdpi.com/2076-3417/12/12/6067

Improvements

ESRGAN (Enhanced SRGAN)

- Improved generator architecture
- Relativistic discriminator
- Superior perceptual quality

Transformer-Based Super-Resolution

SwinIR

- Uses Swin Transformer blocks
- Captures long-range dependencies
- State-of-the-art performance

IPT (Image Processing Transformer)

- Large-scale pretraining
- Unified framework for multiple image restoration tasks

Attention-Based Super-Resolution

RCAN (Residual Channel Attention Network)

- Channel attention mechanism
- Focuses on informative feature maps
- Excellent performance in benchmarks

SAN (Second-Order Attention Network)

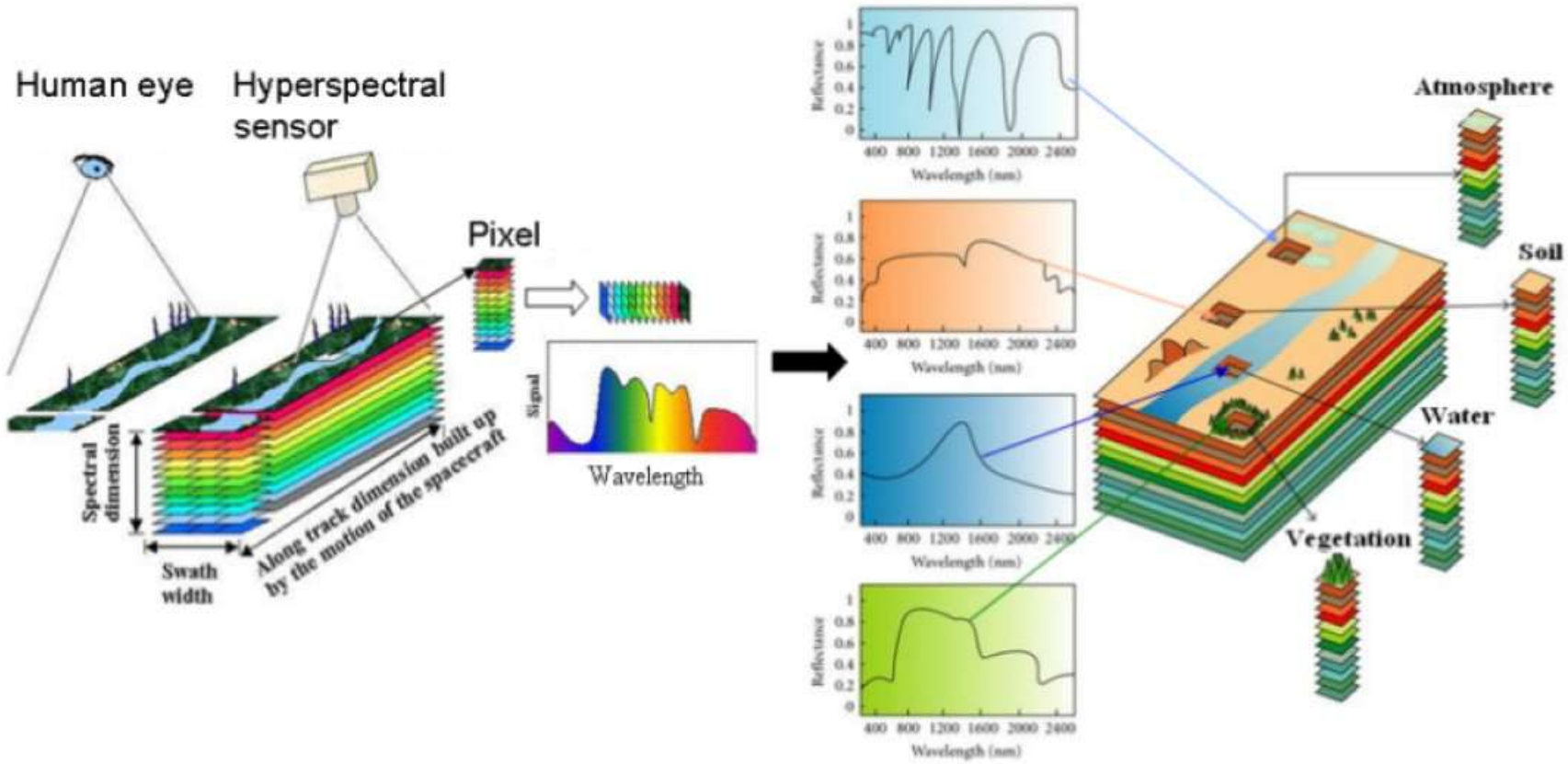
- Uses second-order statistics
- Captures global feature relationships

Applications

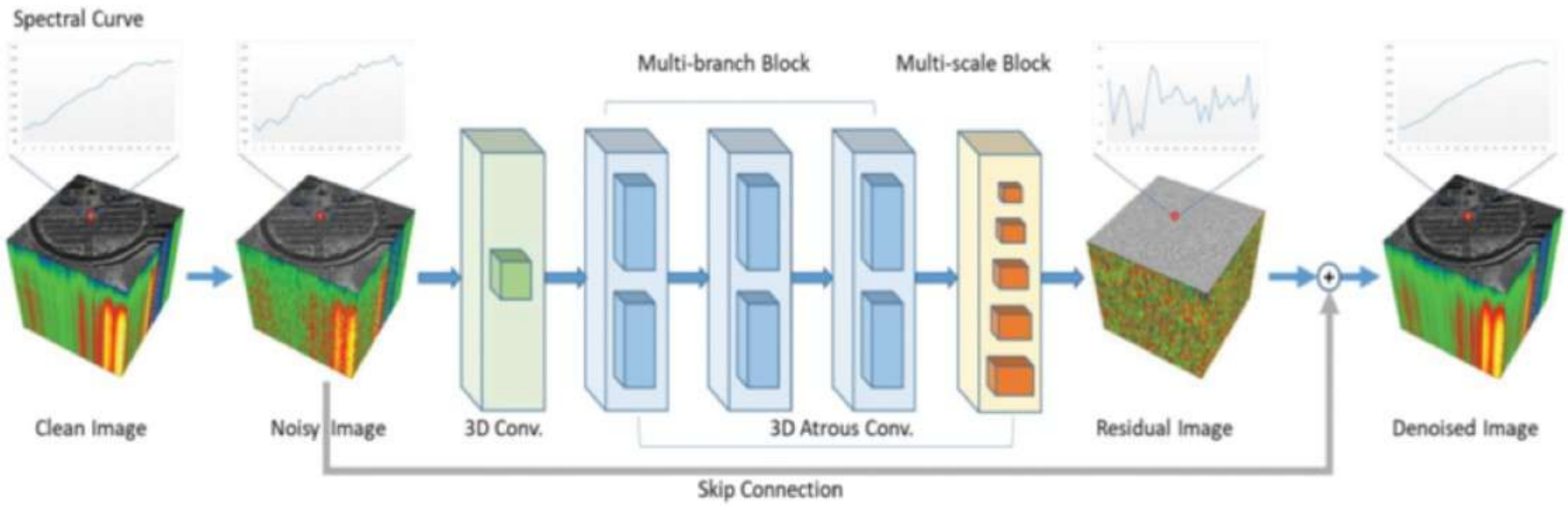
- Medical imaging
- Satellite and remote sensing
- Surveillance and security
- Microscopy
- Consumer photography and video streaming

Hyperspectral image or hypercube or datacube

- A hyperspectral image is an image captured by a sensor that measures electromagnetic radiation across a wide range of wavelengths.
- Unlike traditional images, which typically contain three channels (red, green, and blue), hyperspectral images contain many more spectral bands, often ranging from tens to hundreds or even thousands of bands.

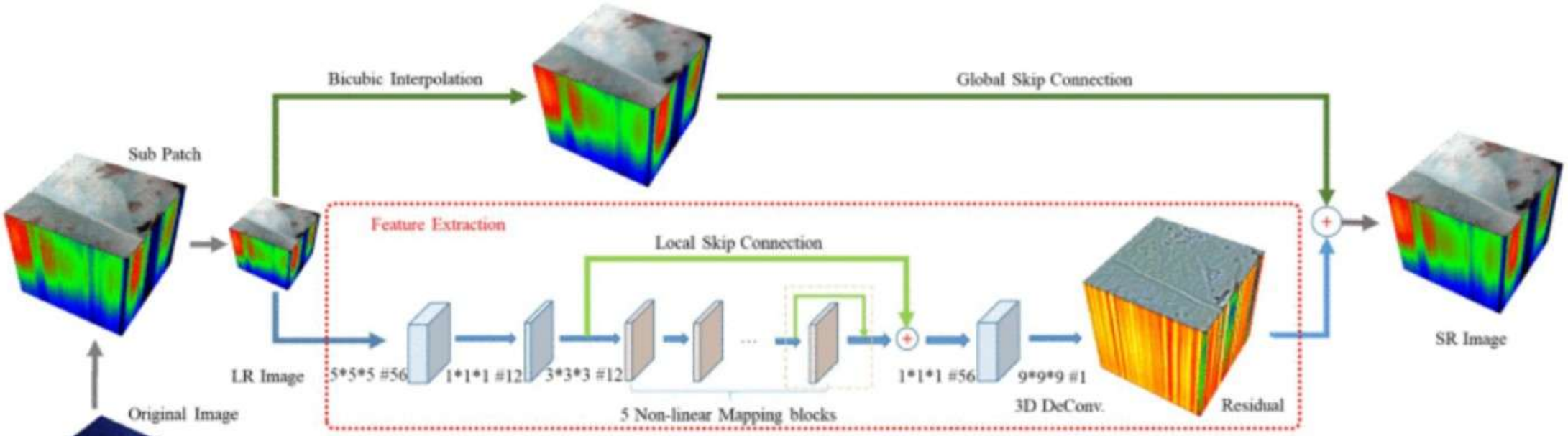


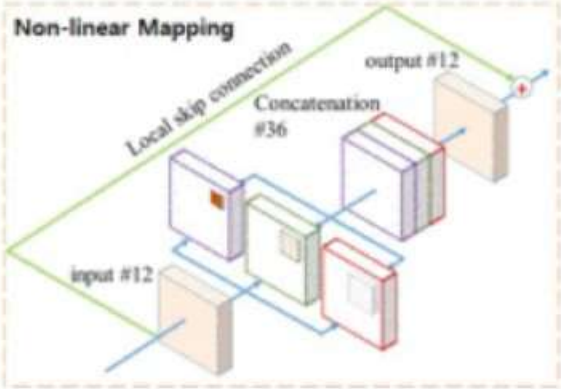
Hyper-spectral Image De-noising

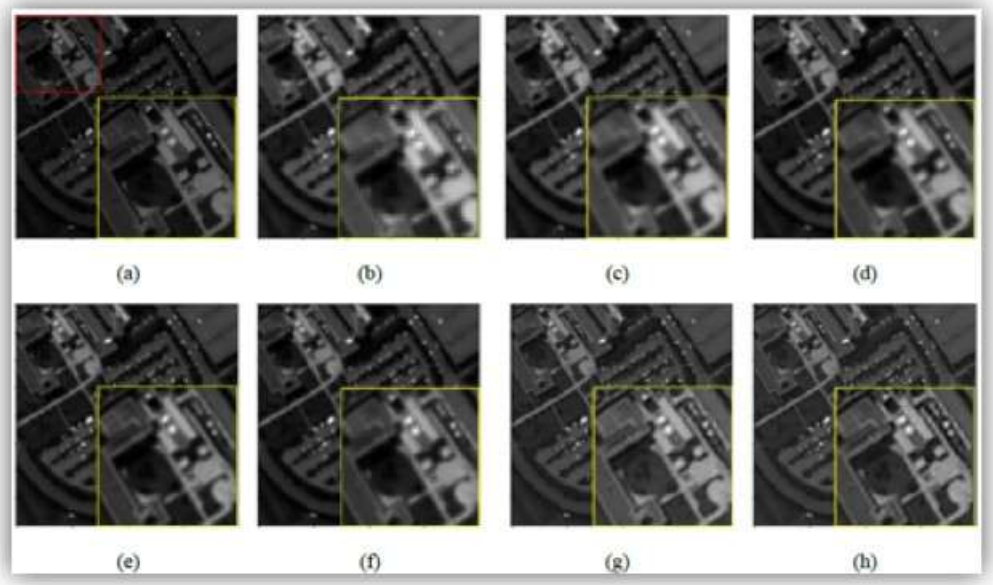


De-noising results: Top: original; Left: 3DADCNN result; Right: 3DWIN5-RB result.

Hyper-spectral Image: Super-resolution







Results: From top left to bottom right (a) Original (b) LR (c) Bicubic (d) 3DVDSR (e) 3DFCNN_zp (f) 3DESP_res (g) 3DFSR_res (h) Ours.

Assignment: Write Report on

1. Synthetic Aperture Radar (SAR)
2. Atmospheric correction models
3. BRDF and reflectance modeling
4. Google Earth Engine (GEE)
5. Quantum remote sensing
6. Global Positioning System
7. Light Detection and Ranging
8. SRGAN
9. ESRGAN (Enhanced SRGAN)
10. RCAN (Residual Channel Attention Network)
11. SAN (Second-Order Attention Network)
12. SwinIR
13. IPT (Image Processing Transformer)
14. Evaluation Metrics: PSNR, SSIM, LPIPS, SAM and ERGAS
15. Loss functions: Pixel-wise loss, and Perceptual loss, Adversarial loss, Total Variation loss
16. Loss functions: Central loss, Loss functions: Triplet Loss
17. Denoising Variational Auto-Encoding
18. Diffusion in Denoising
19. Diffusion for Super-resolution
20. Patch-based Drone image integration
21. Self-supervised Super-resolution Methods

THANK YOU

CONTACT INFO

yagyapandeya@gmail.com
yagy.apandeya@ku.edu.np

+977-9848-542617

https://yagyapandeya.github.io/